CHAPTER 1

OBJECT-ORIENTED DESIGN PRINCIPLES

1 Introduction to SOLID Design Principles 2
2 SOLID: Single Responsibility Principle 3
3 SOLID: Open Closed Principle 5
4 SOLID: Liskov Substitution Principle 8
5 SOLID: Interface Segregation Principle 11
6 SOLID: Dependency Inversion Principle 14
7 Quiz: Object-Oriented Design Principles 16

This chapter covers the SOLID design principles: Single Responsibility, Open Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion.

1 Introduction to SOLID Design Principles

1.1 Introduction

When creating software, we can follow good practices to avoid issues to make our code easier to understand, robust, and maintainable. Few of these practices are often termed as principles, e.g., the **SOLID** principles refer to the best practices to be followed in OOD.
SOLID is an acronym for the first five object-oriented design (OOD) principles by *Robert C. Martin* , also known as *Uncle Bob*, the author of *Clean Code: A Handbook of Agile Software Craftsmanship.*
The illustration below represents the acronym for SOLID design principles.

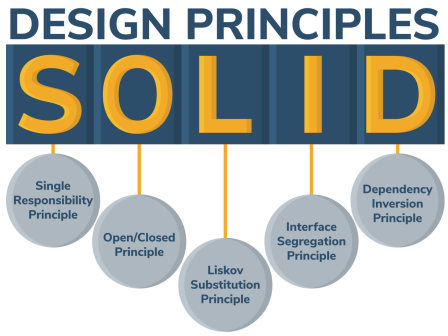


Figure 1.1: The SOLID design principle in object-oriented design

1.2 Why use SOLID principles?

Let's look at the possible issues below that may occur in the code if we don't adhere to the SOLID principles.

- The code may become tightly coupled with several components, which makes it difficult to integrate new features or bug fixes and sometimes leads to unidentified problems.
- The code will be untestable, which effectively means that every change will need end-to-end testing.
- The code may have a lot of duplication.
- Fixing one issue results in additional errors.

However, if we adhere to the SOLID principles, we are able to do the following:

- Reduce the tight coupling of the code, which reduces errors.
- Reduce the code's complexity for future use.
- Produce more extensible, maintainable, and understandable software code.
- Produce the code that is modular, feature specific, and is extremely testable.

1.3 Design principles

Let's look at the definition of the five design principles.

- In the **Single Responsibility Principle (SRP)**, each class should be responsible for a single part or functionality of the system.
- In the **Open Closed principle (OCP)**, software components should be open for extension but closed for modification.
- In the **Liskov Substitution Principle (LSP)**, objects of a superclass should be replaceable with objects of its subclasses without breaking the system.
- The **Interface Segregation Principle (ISP)** makes fine-grained interfaces that are client specific.
- The **Dependency Inversion Principle (DIP)**, ensures that the high-level modules are not dependent on low-level modules. In other words, one should depend upon abstraction and not concretion.

In the next few lessons, we will explain these five design principles in detail.

2 SOLID: Single Responsibility Principle

2.1 Introduction

The **Single Responsibility principle (SRP)** is perhaps the least understood SOLID concepts. Robert C. Martin coined the term and defines it as follows: "*A class should have only one reason to change*." This implies that any class or component in our code should only have one functionality. Everything in the class should be related to just one goal.
When programmers need to add features or new behavior, they frequently integrate everything within the current class. When something needs to be changed later, due to the complexity of the code, the update process becomes extremely time-consuming and tedious. The Single Responsibility Principle helps us create simple classes that perform just one task. This makes making modifications or adding extensions to the existing code much easier.

2.2 Real-life example

The following illustration represents how SRP is applied in real life:

2.3 Book invoice application

Let's try to understand SRP with the help of an example. We have a book invoice application that has two classes: `Book` and `Invoice`. The `Book` class contains the data members related to the book. Whereas , the `Invoice` class contains the following three functionalities:

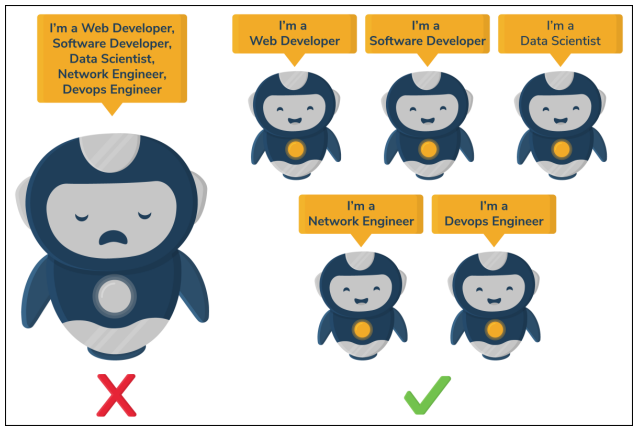


Figure 1.2: An example of SRP

- Calculating the price of the book.
- Printing the invoice.
- Saving the invoice into the database.

The following class diagram provides a blueprint of these classes:

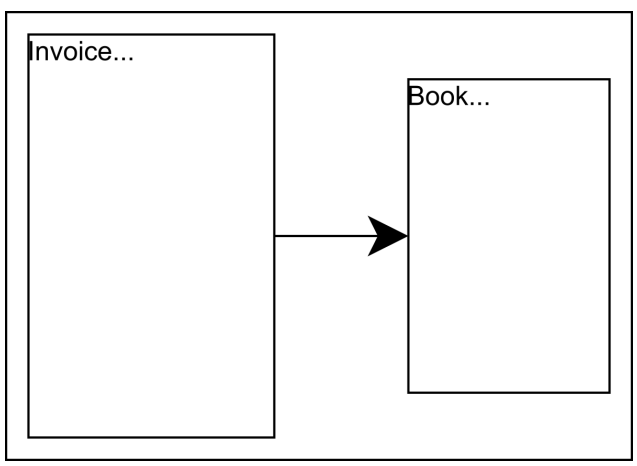


Figure 1.3: The class diagram of the book invoice application without applying the SRP rule

Violations

If we notice, the`Invoice` class violates the SRP in multiple ways:

- The `Invoice` class is about invoices, but we have added print and storage functionality inside it. This breaks the SRP rule, which states, "A class should have only one reason to change."

- If we want to change the logic of the printing or storage functionality in the future, we would need to change the class.

Instead of modifying the `Invoice` class for these uses, we can create two new classes for printing and persistence logic: `InvoicePrinter` and `InvoiceStorage`, and move the methods accordingly, as shown below.

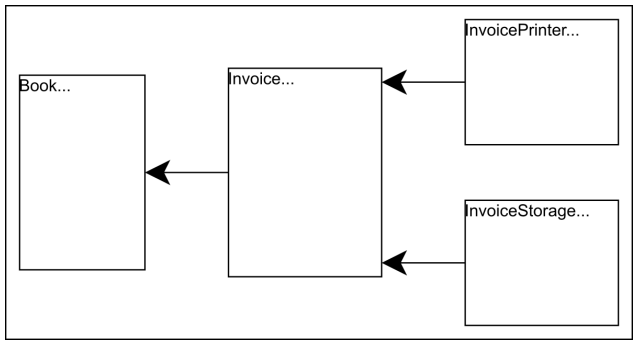


Figure 1.4: The class diagram of the book invoice application after applying the SRP rule

Now, our class structure is in line with the SRP.

2.4 Conclusion

When a class performs one task, it contains a small number of methods and member variables that are self-explanatory. SRP achieves this goal, and due to this, our classes are more usable, and they provide easier maintenance.
In the next lesson, we will learn the Open/Closed design principle with examples.

3 SOLID: Open Closed Principle

3.1 Introduction

In 1988, Bertrand Meyer defined the **Open Closed Principle (OCP)** in the following way, "A software artifact should be open for extension but closed for modification." This means that a system should improve easily by adding new code instead of changing the code core. This way, the core code always retains its unique identity, making it reusable.
One might think of OCP as inheritance, but remember that inheritance is only one of the OCP techniques. We use the interface because it is open for extension and closed for modification. Therefore, OCP is also defined as **polymorphic OCP.**

3.2 Example

Suppose Alex had a cardboard business that sold boxes to its clients. We designed a class for calculating the volume of boxes. It takes the dimensions and calculates the volume of each

box and adds it up to calculate the total volume of all boxes, as shown below.

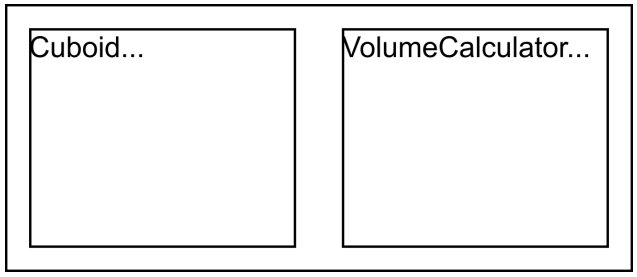


Figure 1.5: The volume calculator class

The algorithm for the `volume(Cuboid)` function is shown in the flowchart below.

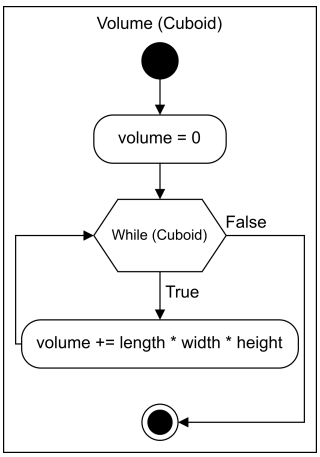


Figure 1.6: The volume(Cuboid) function

As the business grew, Alex also started selling cone-shaped boxes. To integrate the calculation of its volume, we need to make a `Cone` class and update the `volume()` function. See the updated classes below:

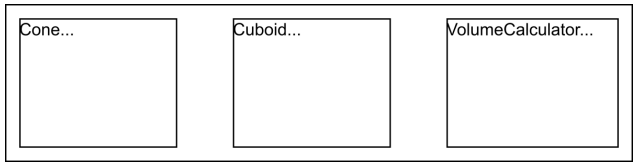


Figure 1.7: Add a Cone class and update the volume method

The algorithm for the `volume(Shape)` function is shown in the flowchart below.

With only two types of boxes, the class structure looks fine, but what if Alex decides to deal with more types of boxes, e.g., a cylinder box? This will add complexity to the `volume(Shape)`. We will divide the code into segments using OCP to overcome this complexity.

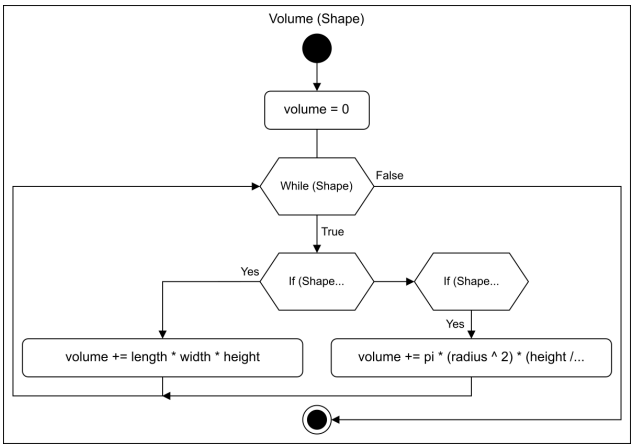


Figure 1.8: The volume (Shape) function

Implementing the Open Closed Principle

We will make a parent class, `Shape`, which is an abstract class and has a `volume()` function, that is extended by its sub-classes, `Cuboid`, `Cylinder`, and `Cone`. These derived classes have their own `volume()` functions according to the shape. Then we have the `VolumeCalculator` class that only performs one task: adding the volume of all the boxes using the `sumVolume()` function.

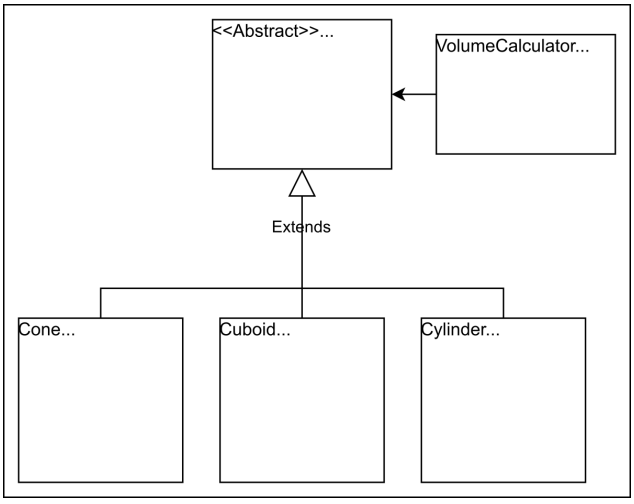


Figure 1.9: The class diagram of VolumeCalculator according to OCP

3.3 Conclusion

We can conclude the OCP discussion as follows:

- A software system should be easy to extend without the need for modification in the existing system. For the software systems, this goal is achieved by OCP.

- The system must be divided into small components, which are arranged, so that core code is always protected from new code.

In the next lesson, we will talk about the Liskov Substitution Principle with its detailed explanation.

4 SOLID: Liskov Substitution Principle

4.1 Introduction

The **Liskov Substitution Principle (LSP)** is one of the fundamental design principles of object-oriented design. The LSP helps guide the use of inheritance in design so that the application does not break. It states that the objects of a subclass should behave the same way as the objects of the superclass, such that they are replaceable. This rule generally applies to abstraction concepts like inheritance and polymorphism.

4.2 The Vehicle class

Let's construct a simple class called `Vehicle` that has some attributes and methods and a subclass `Car` that extends it as shown below:

So far, this implementation seems right since a car IS A vehicle, and the `startEngine()` method will override the superclass method. However, it's not as simple as it looks.

Violation

Let's add a `Bicycle` subclass in this system and see what happens:

This results in a problem. A bicycle is a vehicle, but it does not have an engine. Therefore, the `Bicycle` class should not be allowed to override the `startEngine()` method.

Solution

A possible fix to this issue would be to add two subclasses of `Vehicle` that classify the vehicles as motorized vehicles and manual vehicles as follows:

With this implementation, we have satisfied the LSP.

- `Car` is substitutable with its superclass, `Motorized`, and `Bicycle` is substitutable with its superclass, `Manual`, without breaking the functionality.

- Their methods can also override the methods of the superclass.

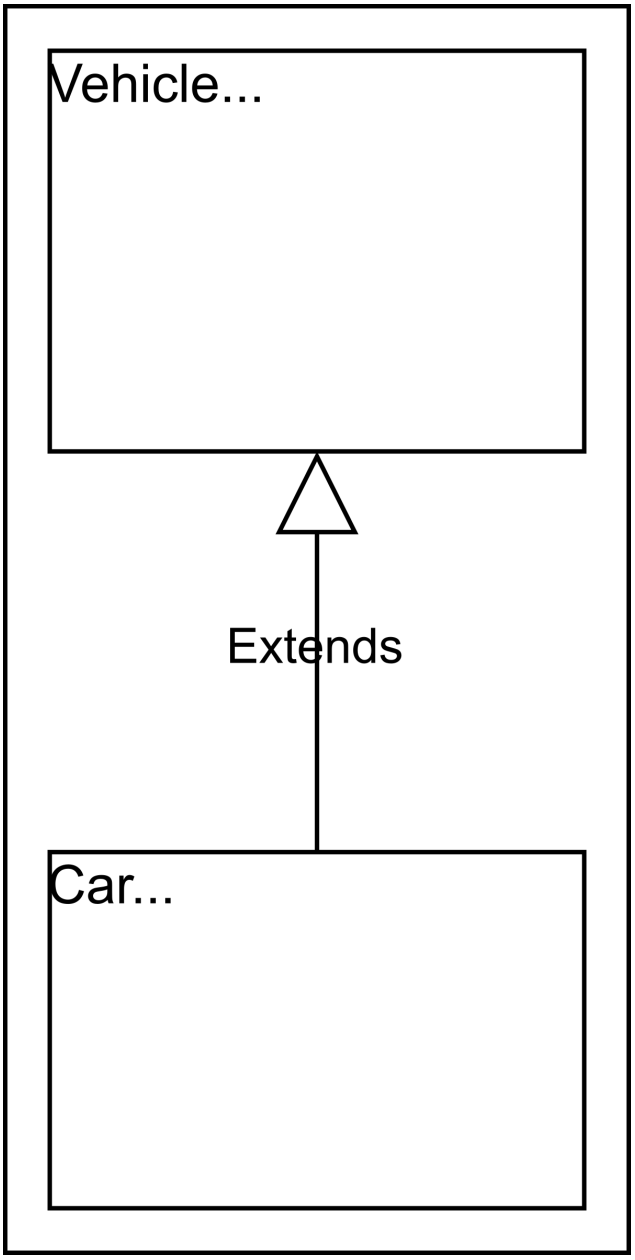


Figure 1.10: The vehicle superclass

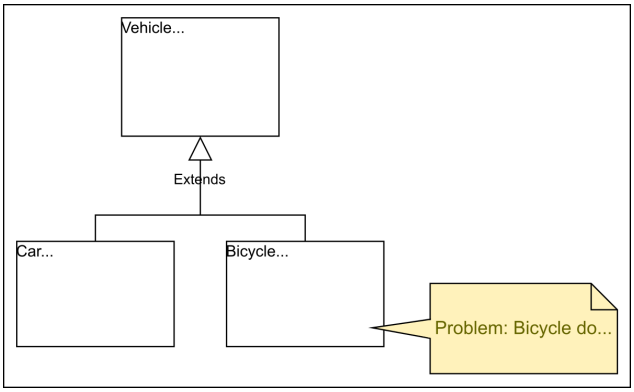


Figure 1.11: Violation of the LSP

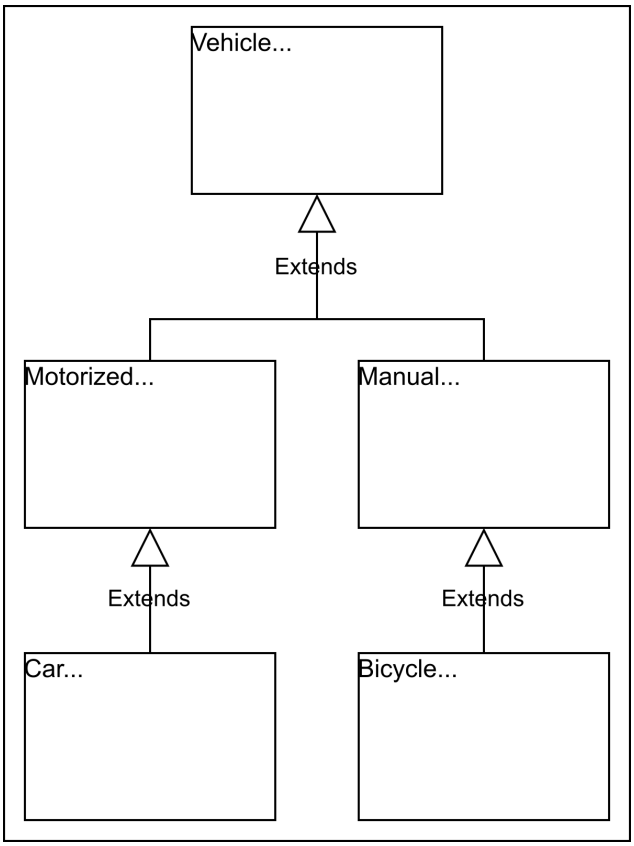


Figure 1.12: An example of the LSP implementation

4.3 Conclusion

The LSP is an important principle that should be extended to the level of system architecture. A small violation of the substitutability of classes can cause the system to break down, which is why we should always be on the lookout for violations. A few benefits of the LSP are provided below:

- It avoids the generalization of concepts that may not be needed in the future.
- It makes the code maintainable and easier to upgrade.

Now that we have learned about the Liskov Substitution Principle, let's look at the Interface Segregation Principle in the next lesson.

5 SOLID: Interface Segregation Principle

5.1 Introduction

The **Interface Segregation Principle (ISP)** is a design principle that does not recommend having methods that an interface would not use and require. Therefore, it goes against having fat interfaces in classes and prefers having small interfaces with a group of methods, each serving a particular purpose.
The goal behind implementing the ISP is to have a precise code design that follows the correct abstraction guidelines and tends to be more flexible, which would help in making it more robust and reusable. This becomes key when more and more features are added to the software, making it bloated and harder to maintain.

5.2 Example

Let's construct a simple interface called `Shape` that has the `area()` method, and `Square` and `Rectangle` as the classes to implement it as shown below:

So far, this implementation seems right, as both the `Square` and `Rectangle` classes are implementing an interface that they're using. Let 's see how this example can violate the ISP.

Violation

Let's add the `volume()` method to the `Shape` interface and have a new subclass `Cube` to implement it:

The violation leads to a problem. The 2-D shapes cannot have a volume, yet they're forced to implement the `volume()` method of the `Shape` interface that they don't have any use of. This is a clear violation of the Interface Segregation Principle.

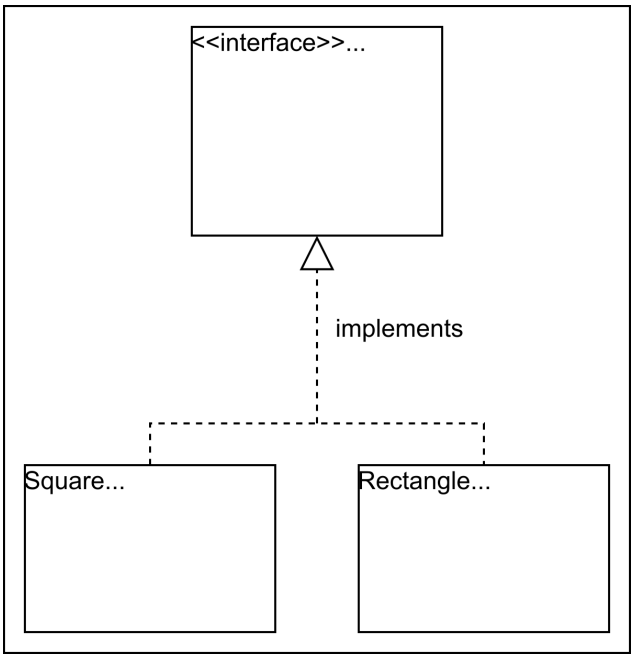


Figure 1.13: The Shape interface

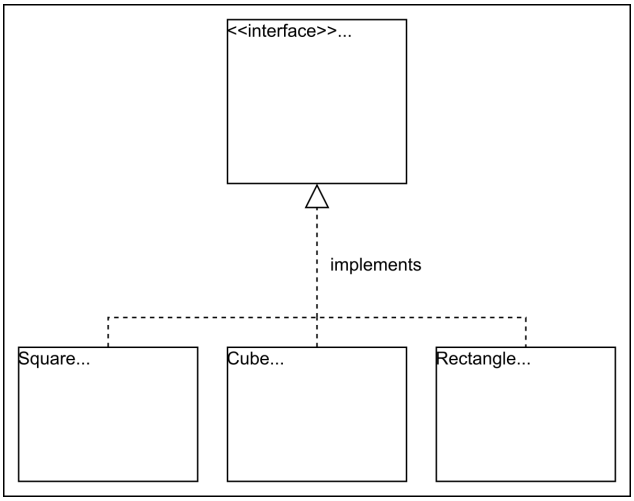


Figure 1.14: Violation of the ISP

Solution

To adhere to the Interface Segregation Principle (ISP), it is essential to ensure that an interface is client-specific rather than general-purpose. In this context, the solution involves implementing the Shape interface into two distinct interfaces: `TwoDimensionalShape` for 2D shapes and `ThreeDimensionalShape` for 3D shapes.

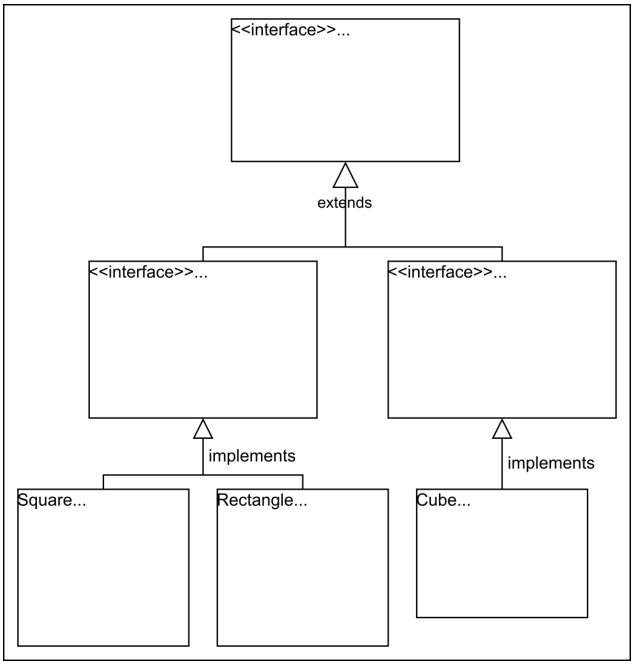


Figure 1.15: Solution of the ISP

By organizing interfaces based on the dimensions of shapes, we avoid forcing 2D shape implementations to provide methods irrelevant to them. The separation follows the Interface Segregation Principle, resulting in a cleaner and more intuitive design. Classes representing 2D shapes only need to implement `TwoDimensionalShape`, while 3D shapes like `Cube` implement `ThreeDimensionalShape`, requiring the implementation of the `volume()` method appropriate to their nature.

5.3 Conclusion

The ISP, being an important principle, is the most violated principle in object-oriented programming. This can easily be achieved by adding more features to our software, requiring us to update large parts of our program. A few benefits of the ISP are as follows:

- It helps to keep our software maintainable and robust.
- It allows for efficient refactoring and redeployment of code.

Let's look at the Dependency Inversion Principle in the next lesson.

6 SOLID: Dependency Inversion Principle

6.1 Introduction

The **Dependency Inversion Principle (DIP)** states that high-level modules should not depend on low-level modules, but rather both should depend on abstractions. The abstractions should not depend on details. Instead, the details should depend on abstractions.
In many cases, thinking about the interaction between modules as an abstract concept allows the linking of components to be reduced without the need for more coding patterns to be implemented. This allows for a functional scheme with reduced implementation and allows the system to be more flexible.

6.2 Real-life example

Let's try to understand the concept of DIP with the help of a school example. Suppose there is a headmaster of a high school. Under the headmaster, there are faculty members such as teachers, assistants, and some helpers.

Violation

Let's see what a possible design would look like without the implementation of the DIP.
The class diagram of this example is shown below:

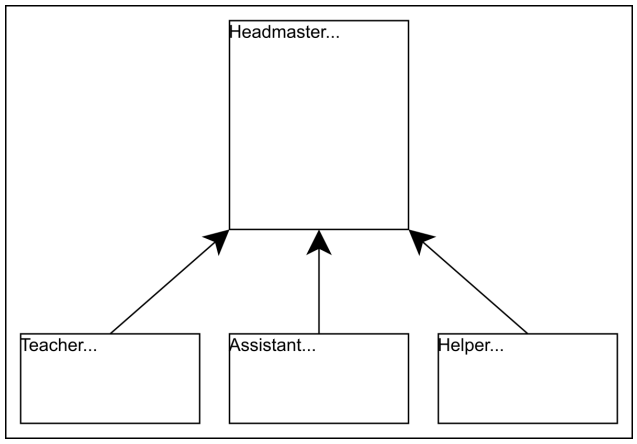


Figure 1.16: Violation of the DIP

Let's note down some issues with this design:

- Everything is exposed from the lower layer to the upper layer, meaning that abstraction has not been implemented. This indicates that the headmaster must know the type of faculty that they can oversee beforehand.
- Now, if an additional type of faculty comes under the headmaster, such as a **secretary**, then the entire system would need to be reconfigured.

Solution

A possible fix to this issue would be to add a `Faculty` class that will be the parent class for all types of faculty. This would reduce the number of dependencies among modules and would make for an easily expandable system.
Let's look at the DIP implemented in class diagram below:

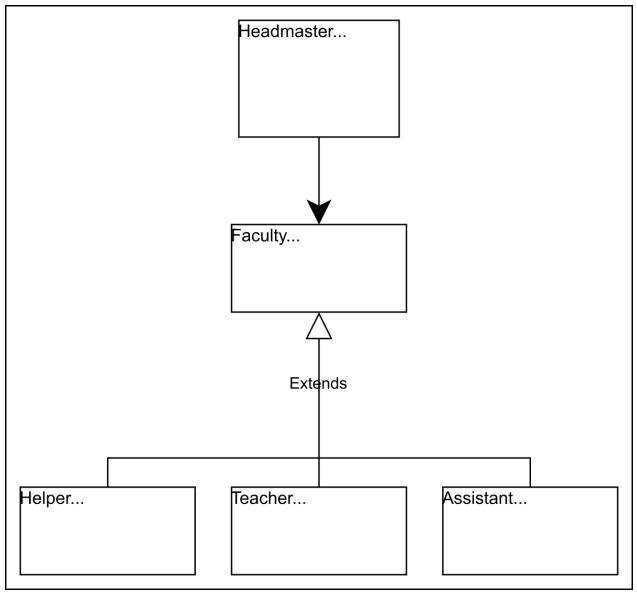


Figure 1.17: DIP example of a high school

Now, if any other kind of faculty is employed, they can just be easily added to the `Headmaster` without the need to explicitly inform the headmaster of it. Let 's take the example of an additional faculty position, `Secretary`. Its class would be a child of the `Faculty` class and would lead to the following diagram:

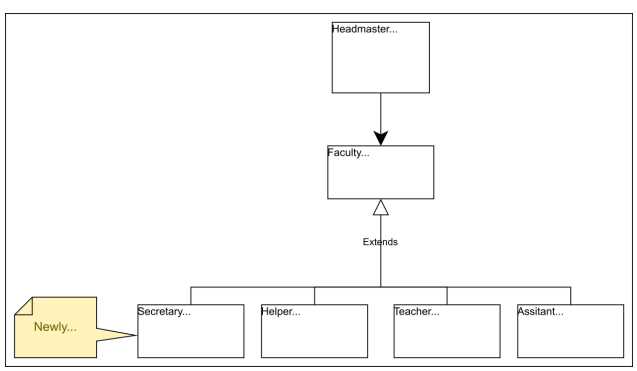


Figure 1.18: Secretary is added to the high school example

With this implementation, we have decoupled some of the modules, and therefore, satisfied the Dependency Inversion Principle.

6.3 Conclusion

The DIP reduces the number of dependencies among modules. It provides a layer of abstraction between lower and higher classes, allowing for changes in the lower class without making changes in the higher class. A few benefits of the DIP are as follows:

- It allows for the flexibility and stability of the software.
- It allows for the reusability of the application modules.

Now that we have gone through all the design principles of SOLID, it is time to test our understanding of OOD principles concepts in the next "Quiz" lesson.

7 Quiz: Object-Oriented Design Principles

7.1 MCQs

Challenge yourself by solving the following quiz questions.

Quiz: Object-Oriented Design Principles

Question 1: What best demonstrates a violation of the Single Responsibility principle (SRP)?

- A class that only handles user authentication
- **[CORRECT]** A class that manages user login, prints invoices, and saves data to a database

  Explanation: The class in option B is taking on multiple unrelated responsibilities—managing login, printing invoices, and saving to a database—violating SRP, which states a class should have only one reason to change.
- A class that calculates the total price of items in a cart
- A class that represents a book with title and author as properties

Question 2: Given a class that prints an invoice and saves it to the database, how should it be refactored to follow the Single Responsibility principle?

- **[CORRECT]** Move printing and saving methods to separate classes.

  Explanation: To follow SRP, each responsibility (printing and saving) should be moved to its own class (e.g., InvoicePrinter and InvoiceStorage), so each class has only one reason to change.

- Add more methods to handle different formats.
- Combine all methods in a utility class.
- Only keep the saving method and remove printing functionality.

Question 3: (True or False)
The Open/Closed principle states that software entities should be open for modification but closed for extension.

- True
- **[CORRECT]** False
  Explanation: The principle states that entities should be open for extension but closed for modification. You should be able to add new functionality without modifying existing code.

Question 4: A developer must add support for a new box shape to an existing volume calculation system without changing the core calculation logic.
Which SOLID principle is most relevant here, and what should the developer do?

- SRP: Add the logic to the main class.
- **[CORRECT]** OCP: Create a new shape subclass and override the volume method.
  Explanation: According to OCP, the system should be extendable (by creating a new shape subclass) without modifying existing code.
- LSP: Change all classes to inherit from a base class.
- DIP: Make all classes depend on concrete classes.

Question 5: What is a sign that the Liskov Substitution principle (LSP) has been violated?

- A subclass adds new optional methods.
- **[CORRECT]** A subclass throws an error when a superclass method is called.
  Explanation: If a subclass cannot properly implement or support a method from its superclass and throws errors when that method is called, it violates LSP.
- A subclass overrides a method with improved performance.
- A subclass adds extra properties.

Question 6: In a vehicle system, adding a `startEngine()` method to the `Vehicle` base class causes a problem when a `Bicycle` subclass is added. What is the best way to resolve this violation of LSP?

- Remove the `Bicycle` class.
- **[CORRECT]** Move `startEngine` to a `MotorizedVehicle` subclass and have `Bicycle` inherit from a different base.

  Explanation: By creating separate hierarchies for `MotorizedVehicle` and `ManualVehicle`, you allow classes to only implement what makes sense for them, respecting LSP.
- Make all vehicles have dummy `startEngine` methods.
- Remove inheritance altogether.

Question 7: Assume we have a `Student` class that contains the database of all the students present in a school.
The `Student` class has two main functionalities as follows:

- Add any new students to the school database.
- Access and display the details of any existing students.

How should the `Student` class be updated to apply the Interface Segregation principle?

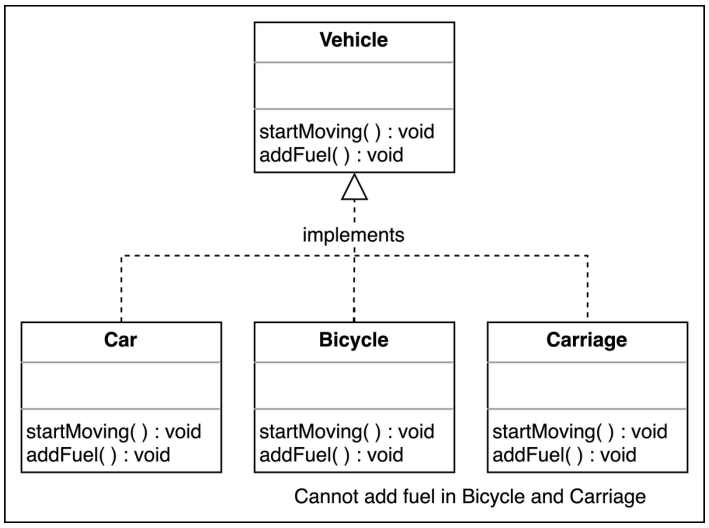
- The `Student` class itself should contain two methods for adding students and accessing students.
- **[CORRECT]** The `Student` class should be divided into two separate classes for adding students and accessing students.
- The `Student` class should only contain the `addingStudents` method. There should be a separate class for accessing students' functionality.
- The `Student` class should only contain the `accessingStudents` method. There should be a separate class for adding students' functionality.

Question 8: (True or False) The Interface Segregation principle recommends having large, general-purpose interfaces so classes can implement as many features as possible.

- True
- **[CORRECT]** False

  Explanation: ISP recommends small, specific interfaces to avoid forcing classes to implement methods they don't need.

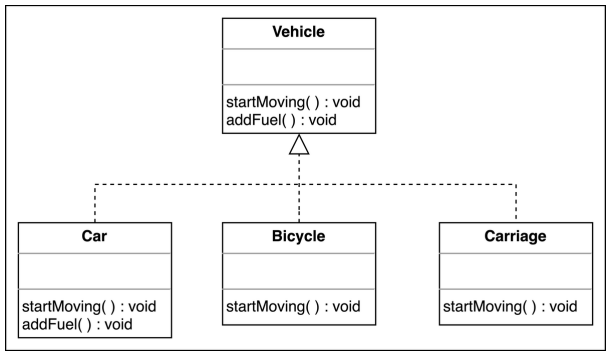
Question 9: An abstract class, `Vehicle`, inherits the classes `Bicycle` and `Carriage` as its subclasses. The `Vehicle` class contains a pure virtual function, `startMoving()`, and is implemented in both subclasses. Suppose we need another vehicle type, such as `Car`, in a system with an `addFuel()` function. Let 's also suppose we decide to inherit the `Car` class from the `Vehicle` class and fulfill the `addFuel()` functionality as shown in the figure:

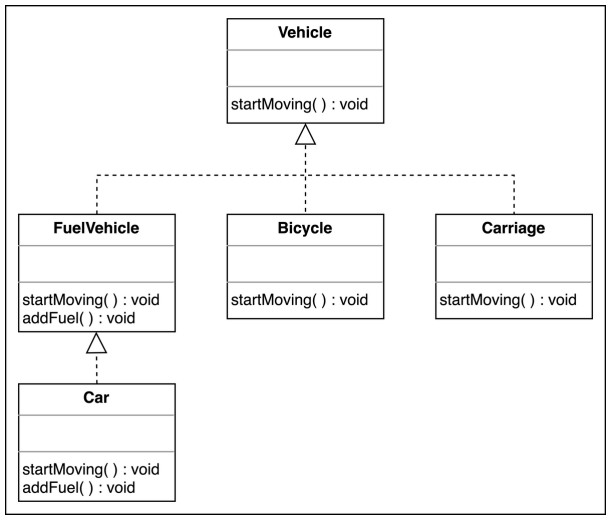


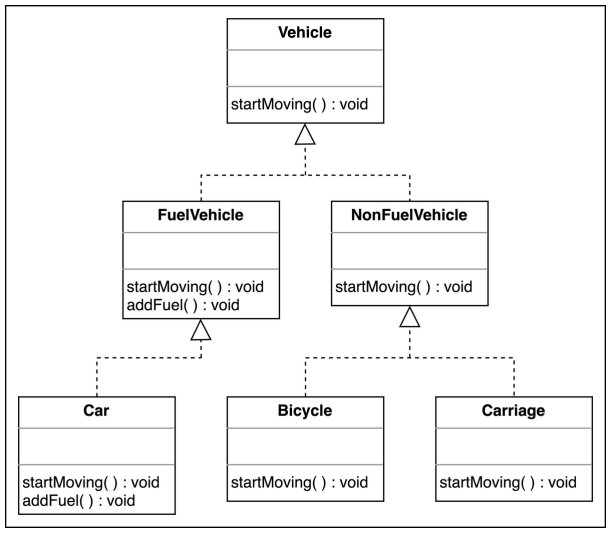
Which design principle is violated by this implementation?

- Single Responsibility principle
- Open/Closed principle
- Liskov Substitution principle
- **[CORRECT]** Interface Segregation principle

Question 10: An abstract class, `Vehicle`, inherits the classes `Bicycle` and `Carriage` as its subclasses. The `Vehicle` class contains a pure virtual function, `startMoving()`, which is implemented in both subclasses. Suppose we need another vehicle type, such as `Car`, in a system that also has an `addFuel()` function.
What is the correct implementation of the above scenario?

- 

- 

- 

- **[CORRECT]** Both B and C